

C29 — Anti-Patterns

1. What interviewers test

"Tell me about a product decision that didn't work." "A product has 40 million registered users and very short sessions; is it healthy?" "Your team shipped on time and the metric didn't move; what do you do?" "What are your common mistakes, and how do you catch them?" Interviewers ask about failure to learn how you reason about it, and because early-career PMs repeat a small set of mistakes. This chapter is not a second list of mistakes; every chapter already has one. It organizes the recurring ones by root cause, gives a *detection signal* for each that you can measure in your own work and mock interviews, and gives a fix with a pointer to where the method is taught. Its use is diagnosis: finding which of these you are most likely to make and catching it early.

Where it shows up. **Inferred**: behavioral questions about failure and learning, execution questions about missed outcomes, and critique questions about why a product failed, at Google, Meta, Microsoft, Amazon, Apple, Netflix, Stripe and others [unverified: candidate-reported]. On the job it is the work of retrospectives and reviews.

What is scored. Whether you can name a failure pattern, say how you would have detected it earlier, and describe a changed practice, without defensiveness **Inferred**. Interviewers listen for specific signals, causes beyond bad luck, a fix that is a habit and not a resolution, and evidence that you apply it.

Six observable behaviors:

1. You name the pattern, not only the event.
2. You identify a root cause that is within your control.
3. You state the signal that would have shown it earlier.
4. You describe a fix as a changed practice with a way to check it.
5. You scan your own work and mocks for your recurring patterns.
6. You do not apply a pattern label to someone else's decision without evidence.

The failure this prevents. Six patterns end failure answers: "it was a timing problem" with no cause, a blamed partner, a lesson that is a platitude, a fix with no check, a list of virtues to avoid vices, and a catalog recited as if it were self-knowledge.

2. The concept from first principles

An **anti-pattern** is a common response to a recurring problem that looks sensible, is easy to adopt and reliably produces a worse result than the alternatives. The word names the response, not the person: good PMs fall into anti-patterns because each one solves a short-term pressure.

C29.1 Five root-cause clusters

Most of the anti-patterns below grow from five roots (FW-C29-02). Knowing the root predicts where else you are likely to slip.

Cluster	Root	What it feels like from inside
A. Activity mistaken for progress	Output is visible and outcomes are slow	"We shipped a lot this quarter"
B. Optimizing the measure	What is measured becomes the goal (Goodhart, C05)	"The number went up"
C. Avoiding discomfort	Disagreement, uncertainty, being wrong and saying no feel costly	"Let's not rock the boat"
D. Pattern-matching instead of reasoning	Borrowing a framework, a competitor or a precedent in place of the situation	"That's what they did"
E. Not closing the loop	No check of whether the decision worked	"We moved on"

Reading the clusters. If your anti-patterns cluster in A, your fix is outcome measures set before work starts; in B, a guardrail and a second metric; in C, scripts and pre-wiring (C32); in D, first-principles questions about the specific user; in E, a review date and a retest (C34.5).

Counterexample. Asked about a failure, a candidate says: "We missed the date because engineering underestimated the work." The interviewer asks what the candidate could have done differently. The candidate has named an event and a cause outside their control, and has no pattern, no signal and no fix. A better answer: "We committed dates for 14 items against about 48 effective engineer-weeks and had 95 weeks of estimates, so we were committed at about twice capacity; the signal I missed was that the plan had no buffer; I now commit to outcomes and ranges and keep 30% unallocated."

When not to use the catalog. Do not use it to label other people's decisions in a meeting ("that's a feature factory"); labels end conversations. Some anti-patterns are right in context: a big-bang launch can be correct for a fixed legal date, and building before testing can be correct when a test costs more than the build. Use the catalog on yourself, and ask whether the context makes the pattern a deliberate choice.

C29.2 Strategy anti-patterns

Anti-pattern	Looks like	Why it happens	Detection signal	Fix (taught in)
Feature factory	The team is judged by features shipped, not outcomes changed	Output is easy to count and to show (A)	Fewer than half of roadmap items have an outcome metric; few shipped items are measured afterward	Attach a metric and a review date to every item; report outcomes (C05, C15)
Vanity metric as goal	Registered users, downloads or page views are the headline	They rise reliably and flatter (B)	The metric rises while retention or revenue is flat; no decision would change if it fell	A metric tied to value, with a guardrail (C05)
Solving a symptom	A fix for a visible problem with no stated cause	Pressure to act; no problem statement (A, D)	No written problem statement; "we need X" with no evidence	The problem-definition template (C02.7)
Competing on a rival's terms	Copying a leader's feature set or strategy	It is safe and easy to justify (D)	The only argument is "they have it"; no segment where you would win	Positioning against a real alternative and a right to win (C18)
Strategy by analogy	"X did it and it worked"	Pattern-matching on survivors (D)	No explanation of why the conditions match; only successes cited	Ask how X differs from us and who failed doing the same (C12 survivorship)
The roadmap as a promise	Dated feature lists shared as commitments	Stakeholders want certainty (C)	Committed work exceeds effective capacity; dates slip and no one is surprised	Commit to outcomes and ranges, with a buffer (C15, C16)
Chasing the loudest customer	The biggest deal or the loudest voice sets the roadmap	Revenue pressure; avoiding no (C)	One account's request outranks data from many; custom work accumulates	The no script with a path to yes (C32.7)
HiPPO decisions	The HiPPO (the highest-paid person's opinion) decides	Deference; no agreed evidence rule (C)	Decisions reverse when a senior person changes their mind; tests are not run on senior ideas	A decision rule agreed before the test; disagree and commit (C14, C32.6)

C29.3 Discovery anti-patterns

Anti-pattern	Looks like	Why it happens	Detection signal	Fix (taught in)
Asking users for solutions	"What features would you like?"	It is easy and feels like listening (A)	Interview notes list requests, not situations or behavior	Ask about the last time they did the job; observe (C06)
Validating with friends or the converted	Feedback from people who like you or already use you	Convenience; survivorship (D)	Sample is mostly known contacts or active users; no one who left or never joined	Recruit from the target segment, including lapsed users (C06, C26)

Anti-pattern	Looks like	Why it happens	Detection signal	Fix (taught in)
Confirmation-seeking research	Questions framed to get yes	Attachment to the idea (B)	Most questions lead; no disconfirming evidence recorded	State what would change your mind first; log disconfirming evidence (C26)
Skipping the riskiest assumption	Testing what is safe to test	The risky test is uncomfortable (C)	The assumption map has no ranked entry that was tested	Rank by impact and uncertainty; test the top first (C26.4)
Interest read as demand	A waiting list or "I'd buy that"	Cheap signals are tempting (B, D)	Sign-ups without price, deposit or repeat use	Move up the commitment ladder (C26.5)
Deciding on tiny samples	A reading from 20 users or one week	Pressure for speed (A)	No interval; effect reversed when the sample grew	Sample size and intervals before the test (C12, C14)
Discovery done once	A study at kickoff, never repeated	Process treated as a phase (E)	The latest user conversation is older than a quarter	A standing weekly user contact (C06)

C29.4 Execution anti-patterns

Anti-pattern	Looks like	Why it happens	Detection signal	Fix (taught in)
Scope creep	Requests added mid-flight with no tradeoff	Saying no is uncomfortable (C)	Scope changes without a corresponding date or cut	Written change requests with the tradeoff (C15.7)
No definition of done or non-goals	"Build it and we'll see"	Ambiguity feels flexible (C)	Disputes at review about whether it is finished	PRD with non-goals and acceptance criteria (C16)
Big-bang launch	Everything to everyone at once	Excitement; pressure of a date (A)	No flag, no staged plan, no rollback trigger	Staged rollout and rollback triggers (C17)
Shipped counted as success	The launch is the milestone	Output is visible (A)	No post-launch review; metrics checked once or never	Success criteria and a review date set before launch (C17, C20.8)
Technical debt ignored until it blocks	Quality work is always next quarter	Features are rewarded more visibly (A, B)	Rising incident rate; estimates growing for similar work	A standing share of capacity and a list of known debt with its cost per sprint (C16.4)
The spec as a handoff	A document thrown over the wall	Writing replaces conversation (A)	Engineering questions arrive after the build starts	Working sessions with engineering and design before and during (C16)
Consensus as the goal	Everyone must agree	Conflict avoidance (C)	Decisions slow; no one owns the call	Decision rights; one decider (C01.1, C32.6)
Over-polishing before learning	Weeks refining a version no one has tried	Perfectionism; fear (C)	No users have seen it after weeks	Smallest testable version; a time box (C15, C26)

C29.5 Communication anti-patterns

Anti-pattern	Looks like	Why it happens	Detection signal	Fix (taught in)
Burying the lede	Background first, point last	Chronology is easy (D)	Readers ask "what do you need from me?"	Answer first (C32.1)
Status as activity	A list of meetings and tasks	It shows effort (A)	No reader can tell the status or the ask	Headline, change, next, need (C32.2)
The surprise in the meeting	A first objection in the room	No pre-wiring (C)	Decisions postponed after the meeting	Pre-wire the objectors one-to-one (C32.3)
All "we"	No personal decision or action visible	Modesty, or hiding (C)	A story with no "I" and no decision	Decision and action in the first person with credit (C31)
Hedging without numbers	"Probably maybe"	Fear of being wrong (C)	No probability, range or check date	The uncertainty statement (C32.8)
Winning the argument, losing the partner	Correct and corrosive	Ego; short-term focus (B)	Partners stop sharing information early	Steelman, agree criteria, commit (C32.6)
Bad news held back	Waiting for a fix before telling	Fear of reaction (C)	Leaders learn from someone else	Tell early with impact and plan (C32.5)

C29.6 Interview-specific anti-patterns

Anti-pattern	Looks like	Why it happens	Detection signal	Fix (taught in)
Framework recitation	A named framework with no user and no decision	Anxiety; memorization (D)	The answer could fit any prompt	Reason from the user and the decision (C03, C07)
Jumping to solutions	Features in the first minute	Eagerness (A)	No user or problem stated before ideas	Clarify, define the user and the problem (C07)
Metrics without guardrails	One success metric	Habit (B)	No counter-metric named	Primary, guardrail and decision rule (C05)
Answering a different question	A prepared story that does not fit	Over-rehearsal (D)	The interviewer re-asks the question	Answer what was asked; say if you pivot (C31)
Inventing under probing	A number that changes	Pressure to look knowledgeable (C)	A figure differs between answers	Say "I don't recall"; give a range (C31.7)
Overclaiming company knowledge	"They obviously did it because..."	Wanting to sound informed (D)	No source for a company claim	Verified, Reconstructed, Opinion labels (C28)

Anti-pattern	Looks like	Why it happens	Detection signal	Fix (taught in)
Practicing the favorite round	Hours on what you already do well	It feels good (A)	Mock scores flat in the weakest dimension	Importance times gap; re-score (C33.14, C34.7)
Reading a bad round as a verdict	Spiraling into the next round	Fear (C)	Scores drop after one weak round	Reset routine; log and fix (C34.8)

C29.7 Detection signals and fixes: the anti-pattern scan

A catalog helps only if you check yourself against it (FW-C29-01). The scan has four steps.

1. **Collect evidence.** Use your artifacts: the last three documents you wrote (a spec, an update, a decision document); your roadmap and its outcome measures; your mock scorecards and error log (C34); and your stories. Evidence beats memory.
2. **Match signals.** For each artifact, run down the detection signals in the tables and mark the ones present. Count how often each appears across artifacts.
3. **Find the cluster.** Group the marked patterns by root cause (A to E). One cluster that dominates is your leverage point.
4. **Fix with a practice and a retest.** Choose one practice (for example, "every roadmap item gets an outcome metric and a review date"), apply it to the next artifact, and check the signal again in three weeks.

Fixes are habits, not intentions. A fix is credible when it names the practice, the artifact where it will appear and the check. "I'll communicate better" fails; "I now send a four-line update with an explicit ask every Friday, and replies rose from 12% to 60%" works (CS-C32-01).

The interview use. When asked about a failure, name the pattern, give its detection signal, describe the practice that replaced it and cite the check. The structure is: *what happened, the pattern, the root cause in my control, the signal I missed, the practice I changed, and the evidence it works.*

3. Frameworks

Three frameworks, set out below. The metric card (FW-C05-01), the experiment decision rules (FW-C14-05), the scope-negotiation script (FW-C15-07), the launch-or-hold table (FW-C17-04), the uncertainty statement (FW-C32-08), the disagreement protocol (FW-C32-07), the story anatomy (FW-C31-03), the answer-journal loop (FW-C34-04) and the weakest-competency diagnosis (FW-C34-06) are the fixes the tables point to and are not repeated.

FW-C29-01 · The anti-pattern scan · Original

- **Inputs:** three or more of your own artifacts (a spec, an update, a roadmap with its outcome measures, mock scorecards and the error log, your stories).

- **Outputs:** the detection signals matched in each artifact, a count by anti-pattern, the dominant root-cause cluster, one practice chosen as the fix with the artifact where it will appear, and a retest date with the signal to recheck.
- **Assumptions:** you scan evidence, not memory.
- **Tradeoffs:** a scan takes an hour and replaces a vague sense of improving with a measured change.
- **Failure modes:** scanning only one artifact; choosing several fixes; no retest.
- **Worked example:** Worked examples 1 and 4.

Artifact	Signals matched	Pattern	Cluster
----------	-----------------	---------	---------

FW-C29-02 · The root-cause clusters · Original

- **Inputs:** a set of marked anti-patterns.
- **Outputs:** each pattern assigned to a cluster (A activity mistaken for progress; B optimizing the measure; C avoiding discomfort; D pattern-matching instead of reasoning; E not closing the loop), the dominant cluster, and the fix that fits it (A: outcome measures first; B: a guardrail and a second metric; C: scripts and pre-wiring; D: first-principles questions about the specific user; E: a review date and a retest).
- **Assumptions:** patterns have a common cause you can address once.
- **Tradeoffs:** clustering simplifies and may misassign a pattern with several causes.
- **Failure modes:** treating each pattern separately; a fix aimed at the wrong cluster.
- **Worked example:** Worked example 4.

FW-C29-03 · The failure-story structure · Original

- **Inputs:** a decision that did not work.
- **Outputs:** an answer in six parts: what happened (with the cost); the pattern; the root cause within your control; the signal you missed; the practice you changed; the evidence that it works.
- **Assumptions:** you can tell it without blame and with numbers.
- **Tradeoffs:** a structured failure story is honest and can sound rehearsed if it has no specifics.
- **Failure modes:** an event with no pattern; a cause outside your control; a fix with no check.
- **Worked example:** Worked example 5.

Part	Content
What happened	The decision and its cost, with a number
The pattern	The named anti-pattern
Root cause in my control	A choice you made, not a circumstance
The signal I missed	What would have shown it earlier

Part	Content
The practice I changed	A habit, with the artifact where it appears
The evidence	A before-and-after measure

4. Answer architecture

The "tell me about a decision that didn't work" answer, for three minutes

Step	Seconds	What to produce
What happened and what it cost	30	The decision, the stakes, one number
The pattern and the root cause	40	The anti-pattern named; a cause in your control
The signal you missed	30	A measurable signal that was available
The practice you changed	40	A habit and the artifact where it shows
The evidence	30	A before-and-after measure; one use since

Pacing rules. Keep it to about three minutes and expect probes: "why didn't you see it?" and "how do you know it works?" Prepare numbers for both.

How to say it. "The pattern was a feature factory: we measured shipping, not outcomes. The signal I missed was that only 6 of 20 roadmap items had an outcome metric."

How to close. "Now every item has a metric and a review date, and last quarter 90% of shipped items were reviewed, up from 21%."

Type: failure, learning or mistakes

The failure-story structure with a named pattern and a measured fix.

Type: why did a product fail

Hypotheses by family (strategy, discovery, execution, communication), evidence for each, the most likely root cause and what you would have measured.

Type: a team is stuck or a metric is flat

Scan for activity-versus-outcome, optimizing the wrong measure, avoided decisions and an unclosed loop.

Type: what are your weaknesses or recurring errors

Your top pattern from a scan, the signal, the practice and its check.

Common interviewer interventions, and what to do

Interviewer says	What it means	What to do
"What could you have done differently?"	Root cause in your control	Name a choice you made and the alternative
"How would you have known earlier?"	A detection signal	A measurable signal that existed
"How do you know it works now?"	Evidence	A before-and-after number
"Isn't that just bad luck?"	Pattern versus event	Name the pattern and show it recurs
"Is that a fix or a resolution?"	Habit versus intention	The practice, the artifact and the check

5. Worked examples

All five examples are **Illustrative**: the teams and figures are invented (invented), and the arithmetic was computed.

Worked example 1 — Scanning a roadmap for a feature factory (Illustrative)

The artifact (FW-C29-01). A team's half-year roadmap has 20 items (invented). Six items carry an outcome metric with a target. Fourteen items have shipped; of those, three have been measured against a target after launch.

Match the signals. *Feature factory*: outcome coverage is 6 of 20 (30%); shipped-and-measured is 3 of 14 (21%). Fewer than half of items have an outcome metric (below the threshold in C29.2), and about four in five shipped items were never evaluated. Of the whole roadmap, 3 of 20 items (15%) have closed the loop.

Read it. The team is organized around output. The cluster is A (activity mistaken for progress), with E (not closing the loop) behind it: items are launched and forgotten, so the team cannot know which of its 14 launches mattered.

The fix (practice and retest). For every new item, write the outcome metric, its baseline and its target in the item before work starts; set a review date four weeks after launch; keep a one-page outcomes log. For the 11 shipped items with no review, run a quick review this month (a metric check, not a project). Retest in six weeks: target at least 90% outcome coverage on new items and at least 80% of launched items reviewed on time.

What this shows. Two ratios from a roadmap turned "we feel busy" into a measured pattern, a cluster and a practice with a retest.

Worked example 2 — A registered-users number and a session distribution (Illustrative)

The situation. A social product reports 40 million registered users. A product manager looks at sessions: of 1,000,000 sessions in a day, 90% last about 3 seconds and 10% last about 300 seconds (invented).

Compute. The mean session length: $(900,000 \times 3 + 100,000 \times 300) \div 1,000,000 = 32.7$ seconds. The median is 3 seconds. The 10% of long sessions account for 30,000,000 of 32,700,000 seconds, about 92% of all session time.

Read it. The mean, 33 seconds, looks like a modest but real product; the median, 3 seconds, shows that most sessions are accidental or abandoned, and a small minority carries nearly all the use. A registered-user count says nothing about this (a vanity metric, C29.2), and an average hides it (survivorship and distribution, C12).

What to measure instead. The share of users with at least three sessions of 30 seconds or more in a week; weekly active users who return in each of four consecutive weeks (a retention curve by cohort, C05); the number of people who create or share something. Percentiles (median, 90th) beside the mean. And the origin of the short sessions: if they come from links or integrations in other products, they are not interest.

Decision. If fewer than, say, 10% of registered users meet the engagement bar, the product's problem is not growth; it is a missing reason to return, and registered-user growth should not be reported as success.

What this shows. A distribution revealed what a mean and a count concealed, and gave a better metric and a decision.

Worked example 3 — The roadmap as a promise (Illustrative)

The situation (FW-C29-01). A team of 5 engineers plans a 12-quarter-week cycle (12 weeks). Capacity is $5 \times 12 = 60$ engineer-weeks. About 20% goes to unplanned work (incidents, support), so effective capacity is 48 engineer-weeks. The roadmap shared with sales commits dates to items whose estimates total 95 engineer-weeks (invented).

Compute. Committed work against effective capacity: $95 \div 48 = 198\%$; against gross capacity, $95 \div 60 = 158\%$. The team is committed to about twice what it can do.

Predict what happens. Dates slip; the team cuts quality or works overtime; sales loses trust; and each slip is explained as bad luck. The pattern is the roadmap as a promise (C29.2), clusters C (avoiding the uncomfortable no) and A.

The fix. Commit to outcomes and ranges, not dated features: share a "now, next, later" view with confidence levels; commit only to about 70% of effective capacity (about 34 engineer-weeks) and hold the rest as buffer; make a published tradeoff when something is added. Where a dated promise is needed (a legal deadline), commit to that item and drop others. Retest: percentage of committed items delivered in the cycle, and the number of date changes communicated late.

What this shows. A capacity calculation made the over-commitment visible before the first slip, and the fix traded certainty about features for certainty about outcomes.

Worked example 4 — Scanning your mock log (Illustrative)

The artifact. Eight mock interviews (invented), with errors tagged from the scorecards and the error log (C34.5). Counts of mocks in which each error appeared: no guardrail metric, 6; no number in the result, 5; ran out of time before risks, 4; jumped to a solution, 3; all "we" in a story, 2.

Compute frequencies. No guardrail: 6 of 8 (75%); no number: 5 of 8 (63%); ran out of time: 4 of 8 (50%); jumped to a solution: 3 of 8 (38%); all "we": 2 of 8 (25%).

Find the cluster. "No guardrail" and "no number" are both cluster B (optimizing the measure, naming a metric without its counter-metric) and cluster E (not closing the loop with a result); "ran out of time" is cluster A (activity, over-long setup); "jumped to a solution" is cluster D. Clusters B and E account for 11 of 20 error instances.

The fix. One practice for the dominant pattern: after naming any primary metric, say its guardrail in the same sentence; and one drill: the metric-definition drill (C34.6), ten reps. Retest in four mocks: target no-guardrail errors in at most 1 of 4 (25%).

What this shows. Counting errors across mocks turned "I'm weak on metrics" into a specific, measured pattern, a cluster and a retestable practice.

Worked example 5 — A failure story, structured (Illustrative)

The prompt. "Tell me about a decision that didn't work." The story and its figures are invented (invented).

The answer (FW-C29-03), as spoken. "Last year I led a quarter for our onboarding team where we shipped fourteen changes and only one moved completion, by about 1.5%. The pattern was a feature factory: we measured shipping, not outcomes. The root cause was in my control: I accepted the roadmap as a list of features and didn't ask for a metric on each. The signal I missed was available: when I counted afterwards, only six of twenty roadmap items had an outcome metric, and only three of fourteen shipped items had been reviewed. What I changed: every item now carries a metric, a baseline, a target and a review date before work starts, and I keep an outcomes log. The evidence: in the next quarter, outcome coverage on new items was 100% and 85% of launched items were reviewed on time; we also stopped two items after the first review because the metric was not moving, which saved about three engineer-weeks. The cost of the original quarter was roughly a quarter of the team's output that did not change a metric."

Probes to prepare. "How did you pick the one that moved?" (it had a metric; I attribute it with a holdout). "Why didn't you see it earlier?" (I looked at velocity, which looked fine). "What would you do if a stakeholder insisted on an item with no metric?" (ask what outcome it serves; propose a lightweight metric and a review date).

What this shows. The six-part structure gave a failure story with a pattern, a signal, a habit and evidence, which survives probing.

6. Baseline, Strong, Standout

Prompt: "Tell me about a time a product decision didn't work."

Baseline answer

"We launched a feature that users didn't adopt. It was a tough market and the timing wasn't right. I learned that I need to listen to users more."

What is wrong: no pattern, a cause outside the candidate's control, no signal, no practice, no evidence.

Strong answer

"We shipped a sharing feature that only 2% of users tried. I had assumed demand from a few customer requests and hadn't tested it. The mistake was building from loud requests without evidence of broader need. What I do now is test the riskiest assumption first with a small prototype before committing engineering time, and I check requests against usage data."

Standout answer

Everything in the Strong answer, plus:

"The pattern was asking users for solutions and treating a handful of requests as demand. The signal I missed was that all six requests came from large accounts, which were 4% of users; I hadn't checked who was asking. My practice changed in three ways: I record, with each request, how many users it represents; I run a smoke test or a manual version before building (the next feature, we tested a fake door for two weeks, and a 0.8% click rate against a 4% threshold stopped us, saving about eight engineer-weeks); and I set a kill criterion before starting. The evidence is that two of my last three proposals were stopped or reshaped at the test stage, and the one we built reached 11% adoption in its first month. The cost of the original feature was about six engineer-weeks and a quarter's goodwill with the team."

What moves an answer up a band

From Baseline to Strong	From Strong to Standout
Owns a cause in their control	Names the pattern and the missed signal with numbers
Describes a changed practice	Shows the practice used since, with evidence and the avoided cost
Keeps it specific	Sets kill criteria in advance; separates request from demand

7. Company lens

All claims describe what companies say publicly or what candidates commonly report, not insider knowledge [as of 2026-10] .

Company lens: Google [as of 2026-10]. Google has publicly shut down a number of products, and its 2018 statement on the consumer version of Google+ (CS-C29-02) is an example of reading engagement honestly Public [V: Google's Project Strobe post of 8 October 2018; closures in CS-C15-02]. Candidates commonly report questions about failures, ambiguity and product decisions with data Inferred. Practical consequence: tell failure stories with a named pattern, a number and a measurable signal, and be ready to discuss engagement distributions.

Company lens: Meta [as of 2026-10]. Meta's published values emphasize moving fast and focusing on long-term impact Public [V: Meta's careers culture page, six core values]. Candidates commonly report questions on speed-versus-quality failures and results with numbers Inferred. Practical consequence: pair "moved fast" stories with the guardrail or staged rollout you added afterward.

Company lens: Microsoft [as of 2026-10]. Microsoft's culture materials emphasize a growth mindset Public [V: Microsoft careers culture page]. Candidates commonly report questions on learning from mistakes and changing your mind Inferred. Practical consequence: show the practice you changed and how you taught it to others.

Company lens: Amazon [as of 2026-10]. Amazon's published principles include ownership and diving deep into data Public [V: Amazon's Leadership Principles, as cited by several summaries]. Candidates commonly report repeated probing of the detail behind a failure Inferred. Practical consequence: have the numbers for the cost, the missed signal and the result, and own the cause.

Company lens: Apple [as of 2026-10]. Apple does not publish a hiring rubric; candidates commonly report an emphasis on craft and judgment about what to leave out Inferred [unverified: candidate-reported]. Practical consequence: include a failure of restraint or over-polishing (C29.4) and the discipline you now apply, without disclosing confidential details of past work.

Company lens: Netflix [as of 2026-10]. Netflix's published culture document emphasizes candor and context Public [V: Netflix culture memo, June 2024]. Candidates commonly report questions on candid feedback and independent judgment Inferred. Practical consequence: tell a failure story that includes the feedback you received and how you acted on it.

Company lens: Stripe [as of 2026-10]. Stripe is widely described as valuing clear thinking and writing Inferred

[V: The Pragmatic Engineer, Inside Stripe's Engineering Culture Part 2 (strong culture of writing); Slab interview with Stripe's former Documentation Manager]. Candidates report questions on rigor Inferred. Practical consequence: write the failure story as a short post-mortem with the six parts of FW-C29-03.

8. Common mistakes

Mistakes in *using* the catalog (the anti-patterns themselves are the subject of section 2).

#	Mistake	How to detect it	Fix
1	Labeling other people's decisions	"That's a feature factory" in a meeting	Use the catalog on yourself; ask questions about their evidence
2	Reciting the catalog as self-knowledge	A list of patterns with no personal example	Scan your artifacts; give one numbered example
3	Naming an event, not a pattern	"We missed the date"	Name the pattern and show it recurs
4	A cause outside your control	"Engineering underestimated"	Find the choice you made
5	A fix that is an intention	"I'll communicate more"	A practice, an artifact and a check
6	Several fixes at once	A list of resolutions	One practice for the dominant cluster
7	No retest	The fix is never checked	A retest date and the signal to recheck
8	Treating a deliberate choice as a pattern	Criticizing a big-bang launch before a legal deadline	Ask whether the context made it right
9	Over-fitting the label	Seeing a pattern in one data point	Count across artifacts
10	Stopping at diagnosis	A scan with no change	Apply the fix to the next artifact

9. Mental models applied

Four models, each tied to a decision in this chapter. [[goodharts-law]] is from C05, [[survivorship-bias]] from C12 and [[inversion]] from C02.

Five whys (MM-five-whys). Ask why repeatedly until you reach a cause you can change [V: Ohno, Toyota Production System, 1988]. "The date slipped" (why?) "we over-committed" (why?) "we had no capacity view" (why?) "I did not ask for one." The last answer is the one a fix can use.

Goodhart's law (MM-goodhart, see [[goodharts-law]], C05). When a measure becomes a target it stops measuring what matters; this is cluster B and the root of vanity metrics.

Survivorship bias (MM-survivorship-bias, see [[survivorship-bias]], C12). Reading only the successes or the active users hides the failures and the leavers; it is behind analogy-based strategy and engagement averages.

Inversion (MM-inversion, see [[inversion]], C02). Ask how you would guarantee a bad outcome and check whether you are doing it: shipping without metrics, validating only with fans, skipping the riskiest test.

Engineering, design and data-science perspectives

- **Engineering.** Engineers see anti-patterns early: shifting scope, specs thrown over the wall, debt deferred. Ask them which of these they see most, and treat their answer as data for your scan.
- **Design and research.** Designers and researchers see discovery anti-patterns: leading questions, friends as participants, interest read as demand. Invite them to review your research plan.
- **Data science.** Data scientists see metric anti-patterns: vanity metrics, averages that hide distributions, tiny samples. Ask them to review your metrics and your reading of results.
- **Your manager and peers.** They see communication and decision patterns that you cannot. Ask for one example of each pattern in your work, and use it in your scan.

Checklist: before you retell a failure or run a retrospective

1. Have I named the pattern, not just the event?
2. Is the root cause a choice within my control?
3. Do I know the signal that would have shown it earlier, and was it available?
4. Is the fix a practice with an artifact where it appears?
5. Is there a retest date and a signal to check?
6. Do I have a before-and-after number and one use since?
7. Did I scan across artifacts, not one data point?
8. Have I asked whether the context made the pattern a deliberate choice?

10. Case studies

CS-C29-01 · Fourteen features and one metric moved · Illustrative · Beginner

Fictional composite. Every number is invented (invented).

Situation. An onboarding team ships fourteen changes in a half-year and celebrates a record output. In the review, the director asks which changes improved completion. The PM cannot say.

Constraints. A roadmap built from stakeholder requests; velocity as the team's headline measure; no standing review of launched work; a PM new to the team.

Data. Of 20 roadmap items, six had an outcome metric with a target; 14 shipped; three of the shipped items had been measured afterward. Completion rose from 52% to 53.5% over the half-year; a holdout on one change attributes about 1.5 points to a single item. The team estimates it spent about 90% of its engineering time on the other 13 items.

Problem definition. As an observable problem: "The team measured shipping, not outcomes, so it cannot tell which of fourteen launches mattered, and only one visibly did."

Analysis. Applying the scan (FW-C29-01): outcome coverage was 30% and shipped-and-measured 21%, the signals of a feature factory (cluster A) with unclosed loops (E). The root cause within the PM's control was accepting the roadmap as a feature list without insisting on metrics. The fix is a practice with an artifact (the roadmap item) and a check (the share of items with metrics and reviews).

Options.

Option	Optimizes	Costs
A. Keep velocity as the headline and add features	Visible output	More unmeasured launches
B. Require an outcome metric, baseline, target and review date on every new item; review the shipped items; stop items whose metric does not move	Evidence of impact	A slower start per item; some items rejected
C. Stop shipping for a quarter to analyze	Learning	Lost momentum

Tradeoffs. B adds an hour per item and shows where effort goes; A repeats the pattern; C is too blunt.

Decision. Choose B. Backfill the metrics for the 11 shipped items without one as a quick review, adopt the item template and an outcomes log, and report outcome coverage and the share of items reviewed alongside velocity.

Outcome. In this fictional scenario, in the next quarter outcome coverage on new items was 100% and 85% of launched items were reviewed on time; two items were stopped after a four-week review because their metrics did not move, freeing about three engineer-weeks (invented).

Alternative outcomes. If stakeholders had resisted metrics on some items, the PM would have labeled them as bets with a lighter metric and a short review date. The lesson: measure the outcome of each launch and close the loop.

CS-C29-02 · Google+, 2018: low engagement, an API bug and the end of the consumer product · Verified · Advanced

Sources (types): Google's post of 8 October 2018 announcing the first findings of Project Strobe and the shutdown of consumer Google+, and its post of 10 December 2018, "Expediting changes to Google+", both read on Google's blog (primary); Google's Workspace blog post of 30 January 2019 on the April 2019 shutdown (primary); and a reference summary of the service's history, which quotes Ars Technica and reports a 2011 user count (secondary). Why Google+ was built and promoted, and how its user counts were compiled, are not stated in the Google posts and are not asserted here. The analysis is the author's. Re-read in October 2026.

Situation. At the beginning of 2018 Google started Project Strobe, a review of third-party developer access to Google account and Android device data. On 8 October 2018 it announced the first findings, among them the shutdown of the consumer version of Google+ through a ten-month wind-down to be completed by the end of the following August [V: Google post, 8 October 2018]. The post said the review had found a bug in one of the Google+ People APIs, discovered and patched in March 2018, under which apps could see profile fields that were shared with the user but not marked public; that the profiles of up to 500,000 Google+ accounts were potentially affected (the company kept the relevant log data for only two weeks); that up to 438 applications may have used the API; and that it found no evidence that any developer was aware of the bug or abused the API, or that any profile data was misused [V: same]. It also said the review "crystallized what we've known for a while": Google+ had not achieved broad consumer or developer adoption, had seen limited user interaction with apps, and the consumer version had low usage and engagement, with 90 percent of user sessions under five seconds; it added that Google+ was better suited as an enterprise product and that the company would focus on its enterprise efforts [V: same]. On 10 December 2018 Google reported a second bug, from a software update in November, that affected about 52.5 million users through a Google+ API; it said it would sunset all Google+ APIs within 90 days and brought the end of consumer Google+ forward from August 2019 to April 2019 [V: Google post, 10 December 2018]. A Google Workspace post of 30 January 2019 gives 2 April 2019 as the date from which consumer accounts would be shut down and content deleted [V: Google Workspace blog, January 2019]. A reference summary reports 90 million users at the end of 2011 and quotes Ars Technica that many sign-ups were incidental to signing up for other Google services [V: reference summary; secondary].

Constraints. A large company with other products a network could be tied to (an inference); larger established social networks as competitors (an inference); a product that had been promoted for years (an inference); a review that found a bug in a Google+ API and, later in 2018, a second bug; and a decision to shut down a product on which, in the post's words, engineering teams had put effort and dedication.

Data. As reported by Google: the first bug and its scale, the second bug, the engagement statement, and the wind-down and acceleration dates; and, from a secondary source, a 2011 user count and a remark that many sign-ups were incidental.

Problem definition. *The author's reading.* A product's reported user numbers and its actual usage diverged, and Google's own review stated the gap publicly in October 2018. As an observable problem: "What does 'users' mean for a product where most sessions end in seconds?"

Analysis. *The author's analysis, through this chapter's tools.*

Vanity metric and distribution (C29.2; Worked example 2). A statement that 90% of sessions lasted under five seconds describes a distribution in which most sessions are almost empty. A count of accounts or registered users cannot show that, and an average session time would be dominated by the small minority of long sessions. The right questions are about the median, the share of users with repeated meaningful sessions and retention by cohort.

Competing on a rival's terms (C29.2). The product competed with larger established social networks (an inference; the Google posts read do not describe the competition); whether the network could ever have reached the level needed for people to find value is a network-effect question (C18, C19).

What kept it going. The Google posts read do not say why the product was kept for years or how its user counts were compiled. A secondary report quotes Ars Technica saying many sign-ups were incidental to signing up for other Google services, which is the question to ask of any such number: does it count intentional use or incidental exposure?

Closing the loop (C29.1, cluster E). The post says the review "crystallized what we've known for a while" and presents the API bug as underlining the difficulty of maintaining the product; the sources read do not say whether the bug changed the timing of the decision, whether a kill criterion existed, or when low engagement was first recognized inside the company. The December post says that a second bug brought the closure forward.

What the evidence does not show. The company's internal metrics before 2018, its reasons for continuing the product, or the cost of keeping it. The case supports an analysis of how a headline number can hide the real distribution and how decisions to stop get delayed, not a verdict on the people involved.

Options. *The author's reconstruction of choices a team could face; not a record of Google's deliberations.*

Option	Optimizes	Costs
A. Keep investing to grow users	A chance at network effects; avoids admitting failure	Continued cost with weak evidence of use
B. Set a kill criterion early on engagement (for example, share of users with repeated meaningful sessions) and review it on a date	Early honest learning	Earlier, more visible failure
C. Narrow to a segment with real use (here, business use, which Google said it would keep) and stop the consumer service	Preserves what works	A smaller product

Tradeoffs. B and C cost an earlier admission and save years of spend; A preserves hope and, per the evidence, a low-engagement product.

Decision. As documented, the company announced the shutdown of the consumer version in October 2018 and said it would focus on enterprise use [V: Google post, 8 October 2018].

Outcome. Reported: a wind-down announced in October 2018 for completion by August 2019, brought forward to April 2019 after a second bug, with consumer accounts shut down from 2 April 2019 [V: Google posts, October 2018, December 2018 and January 2019]. What the company learned and what resources it freed are not described in the sources read.

Alternative outcomes. *Inference.* With an engagement-based kill criterion set at launch, the product might have been narrowed or stopped years earlier. A reader should avoid hindsight and unfairness: many products struggle against a dominant rival, and a company that shuts one down is making a normal decision. The method to carry away: look at the distribution behind a headline number, ask what counts as a user, set kill criteria in advance, and review them on a date.

11. Interview questions and model answers

Eight questions. Model answers are *Illustrative* (composite candidates), and every figure is invented (invented). Each ends with follow-ups and answers. No company tag is attached because the topic is not tied to a single company.

IQ-C29-01 · Beginner · execution · EX

Prompt. What is a feature factory, and how would you avoid being one?

Rubric.

- **Baseline:** "A team that ships too many features."
- **Strong:** A team judged by output instead of outcomes, with a fix.
- **Standout:** Names detection signals (outcome coverage, shipped-and-measured share), explains the root cause, and describes practices with a check.

Model answer. "A feature factory is a team that is rewarded for shipping features, not for what the features change. It happens because output is visible and outcomes are slow. I'd look for two signals: the share of roadmap items that have an outcome metric and target (if under half, it's a warning), and the share of shipped items that were measured afterward. If 6 of 20 items have metrics and 3 of 14 shipped items were reviewed, the team cannot tell which launches mattered.

"To avoid it: every item gets a metric, a baseline, a target and a review date before work begins; I keep an outcomes log; I report outcome coverage and on-time reviews beside velocity; and I stop items whose metric doesn't move after review. The check is a number: in the next quarter I'd want near 100% coverage on new items and over 80% of launches reviewed on time."

Signals. The definition; measurable signals; practices with a check.

Follow-ups.

- **Q.** What if stakeholders want a feature without a metric? **A.** "I'd ask what outcome it serves and propose a light metric and a short review date. If it truly has no outcome, I'd label it a bet, time-box it and say so."
- **Q.** Isn't shipping fast valuable? **A.** "Yes, when it is fast learning. The anti-pattern is shipping without checking what happened. A fast team with reviews is the goal."

IQ-C29-02 · Beginner · execution · EX

Prompt. What is a vanity metric? Give an example and a better alternative.

Rubric.

- **Baseline:** "A metric that looks good but means nothing."
- **Strong:** One that rises without informing decisions, with an example.
- **Standout:** Gives a test (would a decision change if it fell?), an example with the better metric and a guardrail, and notes the role of distributions.

Model answer. "A vanity metric is a number that rises reliably and flatters but does not tell you whether the product is working or what to do. My test is: if this number fell, would any decision change? Registered users is the standard example: it only grows. A better metric is weekly active users who return in each of the last four weeks, by cohort, which shows whether people keep coming back. I'd pair it with a guardrail such as complaints or cancellations so that it cannot be raised by tricks."

"Averages can also be vanity: a mean session length of 33 seconds can hide that 90% of sessions are 3 seconds. So I'd report the median and a share-above-threshold, such as the share of users with three meaningful sessions a week."

Signals. A decision test; a better metric with a guardrail; distributions.

Follow-ups.

- **Q.** Is registered users always useless? **A.** "No: it is the top of a funnel and useful for capacity or cost planning. It becomes a vanity metric when it is the goal."
- **Q.** How do you know your replacement isn't vanity too? **A.** "Apply the same test: does a fall change what we do, and can it be raised without raising value? If it can be gamed, add a guardrail or a second metric."

IQ-C29-03 · Intermediate · execution · EX

Prompt. A senior stakeholder insists you build a feature because a competitor has it. What do you do?

Rubric.

- **Baseline:** "Build it to stay competitive," or "refuse."
- **Strong:** Asks about the user and the evidence.

- **Standout:** Separates strategy by analogy from a real gap, checks who is leaving and why, sizes the gap, offers a test, and uses a decision rule while respecting the decider.

Model answer. "I'd take the concern seriously and ask for the evidence. 'They have it' is strategy by analogy; the questions are whether customers are leaving because we lack it, how many, and what they say. I'd check churn reasons, lost-deal notes and usage of the competitor's feature if public, and ask who would use it and what job it does for them. I'd also ask how the competitor's situation differs from ours, and who has tried it and failed.

"Then propose a proportional step: a smoke test or a manual version, with a threshold agreed in advance, and a decision date. If the evidence shows a real gap, build it; if not, document why and keep watching. If the stakeholder decides to build it anyway and it's within their authority, I'd commit, write down my concern and the signal that would make us revisit, and measure the result."

Signals. Evidence over analogy; a test with a rule; commitment after the decision.

Follow-ups.

- **Q.** What if they say there's no time to test? **A.** "I'd offer the cheapest evidence available that day: a look at lost-deal reasons and a few customer calls. The cost of a week of checking is small against months of building the wrong thing."
- **Q.** What if the feature works for the competitor? **A.** "Then I'd learn why: their users, their positioning, their data. I'd borrow the principle, not the feature, and test it for our segment."

IQ-C29-04 · Intermediate · execution · EX

Prompt. Users keep asking for a feature. Do you build it?

Rubric.

- **Baseline:** "Yes, users know what they want."
- **Strong:** Asks about the underlying need.
- **Standout:** Separates requests from situations, counts who is asking and what share they represent, checks behavior, considers alternative solutions, and tests before building.

Model answer. "I'd treat the request as a symptom of a job, not as a specification. I'd ask: what are they trying to do when they want it, what do they do today, and how often? I'd count how many are asking and what share of users and revenue they are, because a loud ten may not represent a quiet thousand. I'd check behavior: do those users already work around the problem, and where do they drop?

"Then I'd list other ways to meet the need (a setting, a default, a different feature) and test the cheapest one: a fake door to see how many click, or a manual version for five customers. I build the request as asked only if it turns out to be the best way to serve the need and the numbers justify it."

Signals. Request versus job; counting and representation; alternatives; a cheap test.

Follow-ups.

- **Q.** What if the requesters are your biggest customers? **A.** "Their needs matter, and I'd check whether the need generalizes. If it is specific, a configuration or a priced custom solution may be better than a core feature. I'd say so with the numbers and offer a path to yes."
- **Q.** What if the test shows few people click? **A.** "Then demand is lower than the requests suggested, and I'd tell the requesters what I learned and what I can do instead. I'd record the result so the next request is judged against it."

IQ-C29-05 · Intermediate · execution · EX

Prompt. Your launch hit its date, but the metric did not move. What do you do?

Rubric.

- **Baseline:** "Wait and see," or "ship another feature."
- **Strong:** Checks adoption and measurement.
- **Standout:** Diagnoses along the funnel (awareness, activation, value), checks the metric and holdout design, reads effects by week and segment, decides to iterate, stop or extend, and records the learning.

Model answer. "First check that the measurement is sound: is the metric computed correctly, is there a holdout, and is the sample large enough? Then diagnose along the path: did users find the feature (awareness), try it (activation), and get value (repeat use)? If adoption is 4% against a 15% goal, the problem is awareness or activation, with different fixes: placement and messaging, or a simpler first use. If adoption is fine and the metric did not move, the feature may not affect the outcome, and the assumption linking them was wrong."

"Read by week and segment: a novelty bump that faded, or an effect in one segment that is diluted in the average. Decide by the criteria set before launch: iterate if there is a fixable path, stop if the assumption failed (and free the capacity), extend the test if it is underpowered. Then record the learning in the outcomes log."

Signals. Measurement check; funnel diagnosis; segments and weeks; a decision rule; a record.

Follow-ups.

- **Q.** What if you had no holdout? **A.** "I'd use before-and-after with caution, compare with similar periods, and look for a natural comparison group. I'd also add a holdout for the next launch."
- **Q.** How long do you wait? **A.** "As long as the review date I set: long enough for the effect to settle and a full weekly cycle, usually several weeks; I'd not extend indefinitely without a reason."

IQ-C29-06 · Intermediate · behavioral · BE

Prompt. How do you spot your own recurring mistakes?

Rubric.

- **Baseline:** "I reflect after projects."
- **Strong:** Reviews artifacts and asks for feedback.
- **Standout:** Runs a scan on artifacts with detection signals, counts patterns, finds the root-cause cluster, chooses one practice with a retest, and uses outside examples.

Model answer. "I scan evidence, not memory. I take my last three specs, my roadmap with its outcome measures, my mock or review scorecards and my stories, and check each against a list of detection signals: for example, does each roadmap item have a metric, did I state the guardrail, did the story show a decision. I count how often each pattern appears, group them by root cause (activity mistaken for progress, optimizing the measure, avoiding discomfort, pattern-matching, not closing the loop), and look for the dominant cluster.

"Then one practice, not five: say, 'I name the guardrail in the same sentence as the metric.' I apply it to the next artifacts and recheck in three or four weeks. I also ask a teammate for one example of each pattern they've seen me do, since I can't see all of them. Last time my scan showed that in 6 of 8 mocks I gave no guardrail; after the practice and ten drills, it was 1 in 4."

Signals. Evidence; counts; a cluster; one practice and a retest; outside views.

Follow-ups.

- **Q.** What if the scan finds many patterns? **A.** "I'd pick the cluster they share and one practice for it. Fixing the root often removes several symptoms."
- **Q.** How do you avoid defensiveness when a colleague points one out? **A.** "I ask for a specific example and the effect on them, thank them, and add it to the scan. A pattern named by someone else is data I could not collect alone."

IQ-C29-07 · Hard · execution · EX

Prompt. A product has 40 million registered users, but most sessions are very short. Is it healthy?

Rubric.

- **Baseline:** "40 million users is great."
- **Strong:** Questions the registered number and asks about engagement.
- **Standout:** Computes how mean and median differ, defines meaningful use, asks how sessions arise (intent or incidental), reads cohorts and retention, and sets decision criteria.

Model answer. "Not necessarily. Registered users counts everyone who ever signed up and says nothing about use. If 90% of sessions last about 3 seconds and 10% last about 300, the mean is 32.7 seconds, which looks fine, but the median is 3 seconds and 92% of the time comes from 10% of sessions. So I'd measure the median and the share of users with repeated meaningful sessions, say three sessions of at least 30 seconds a week, and retention by cohort.

"I'd ask how short sessions arise: from users who intend to be there, or from links, notifications or integrations in other products that land them there? Incidental sessions are not interest. I'd also check who the long sessions are: a loyal minority is a possible niche; no one is a problem. Decision criteria: if fewer than, say, a tenth of registered users meet the engagement bar and cohort retention falls to near zero, the issue is not growth but a missing reason to return, and I'd narrow to the segment with real use or stop."

Signals. Distribution; meaningful use; origin of sessions; cohorts; a decision rule.

Follow-ups.

- **Q.** What if the company has a strategic reason to keep the product? **A.** "Then it should be explicit and bounded: say what the product is for (for example a shared identity service), measure that, and stop treating consumer engagement as the goal. An unstated strategic reason hides failing measures."
- **Q.** How would you decide when to stop? **A.** "With kill criteria set in advance on engagement and retention, reviewed on a date, with a narrower alternative considered first. Stopping is cheaper when it is planned."

IQ-C29-08 · Hard · behavioral · BE

Prompt. You discover your team has been optimizing a metric that does not matter for the last six months. What do you do?

Rubric.

- **Baseline:** "Switch metrics and move on."
- **Strong:** Tells the team and leadership and resets the goal.
- **Standout:** Verifies the finding, owns your part, quantifies what was lost, tells stakeholders early with a plan, reconstructs which work still matters, sets the right metric with a guardrail and a review date, and changes the practice that allowed it.

Model answer. "First verify: what exactly doesn't matter, and what is the evidence (no relation to retention or revenue in cohorts, say). Then tell my manager and the team promptly, in person and then in writing: here is what we found, what it cost (say, six months of two-thirds of our effort), what I think we should do and when I'll update. I'd own my part, since I helped set the goal, without self-flagellation."

"Then reset: choose the metric that tracks value (a retained-user measure, with a guardrail), set a baseline and a target, and review the past six months of work against it to see what still helps: some of it likely does. Decide what to stop, keep and reshape, with numbers. And I'd change the practice: every goal metric gets a documented link to an outcome, checked quarterly against cohorts, so this can't recur silently."

Signals. Verification; early disclosure with a plan; ownership; salvage; a changed practice.

Follow-ups.

- **Q.** How do you tell leadership without damaging trust? **A.** "Early, factual and with a plan. Leaders react worse to learning it from someone else. I'd say what we know, what we don't, and the decision I need from them."
- **Q.** What if the team resists changing? **A.** "I'd show the evidence and the cohort chart, involve them in choosing the new metric, and recognize the work that remains valuable. People accept a reset more easily when the learning is credited to the team."

12. Exercises

Eight exercises. Each has a model solution and an alternative. Every figure is invented.

EX-C29-01 · Classify the scenarios

Problem. For each scenario, name the anti-pattern, the cluster (A to E), the detection signal and a one-line fix: (a) the team celebrates 14 launches and cannot say what changed; (b) the PM tests a new idea only with friends and colleagues; (c) a PM sends a 500-word weekly update listing meetings; (d) a stakeholder sets the roadmap in a meeting and no one challenges it; (e) a candidate names a framework and applies it to every prompt; (f) a team reports registered users as its north star.

Context. Using the catalog.

Data. None.

Constraints. Eight minutes.

Expected approach. C29.2 to C29.6.

Solution. (a) Feature factory; A/E; low outcome coverage and shipped-and-measured share; metrics and review dates on every item. (b) Validating with friends; D/B; sample is known contacts; recruit from the target segment, including lapsed users. (c) Status as activity; A; no reader can state the status or the ask; headline, change, next, need. (d) HiPPO decision; C; no agreed evidence rule; a decision rule and disagree-and-commit. (e) Framework recitation; D; the answer would fit any prompt; reason from the user and the decision. (f) Vanity metric as goal; B; the metric rises while retention is flat; a value-linked metric with a guardrail.

Alternative solution. Add a second possible pattern for each and say which evidence would decide.

Tradeoffs. One scenario can show several patterns; pick the one the evidence supports.

EX-C29-02 · Roadmap ratios

Problem. A roadmap has 30 items; 9 have an outcome metric; 22 have shipped; 5 of the shipped items were reviewed after launch. Compute outcome coverage, the share of shipped items reviewed and the share of the roadmap with a closed loop. What do you conclude and do?

Context. Feature-factory scan.

Data. As above.

Constraints. Five minutes.

Expected approach. Worked example 1.

Solution. Outcome coverage: $9 \div 30 = 30\%$. Shipped items reviewed: $5 \div 22 = 23\%$. Closed loop across the roadmap: $5 \div 30 = 17\%$. The team is output-oriented (A) and does not close loops (E). Fix: require a metric, baseline, target and review date on new items; run quick reviews of the 17 unreviewed shipped items; report coverage and on-time reviews. Retest in six weeks: at least 90% coverage on new items and 80% on-time reviews.

Alternative solution. Set the threshold by risk: items touching the core flow must have metrics; low-risk fixes may not.

Tradeoffs. Metrics for every small change create overhead; scale them to the stakes.

EX-C29-03 · Mean and median

Problem. In a day, 800,000 sessions last about 4 seconds and 200,000 last about 120 seconds. Compute the mean session length, the median and the share of total session time in the long sessions. What would you report instead of the mean?

Context. Distributions behind a headline number.

Data. As above.

Constraints. Five minutes.

Expected approach. Worked example 2.

Solution. Mean: $(800,000 \times 4 + 200,000 \times 120) \div 1,000,000 = 27.2$ seconds. Median: 4 seconds (80% of sessions are short). Long sessions' share of time: $24,000,000 \div 27,200,000 = 88\%$. Report the median, the 90th percentile, the share of users with at least three meaningful sessions a week and weekly retention by cohort. Ask how the short sessions arise.

Alternative solution. Report the distribution as a histogram with buckets of seconds.

Tradeoffs. One number is easy to track and hides the shape; two or three show it.

EX-C29-04 · Capacity and promises

Problem. A team of 4 engineers has a 10-week cycle. About 25% of time goes to unplanned work. The roadmap commits dates to items estimated at 66 engineer-weeks. Compute effective capacity and the commitment ratio; propose a commitment policy.

Context. Roadmap as a promise.

Data. As above.

Constraints. Six minutes.

Expected approach. Worked example 3.

Solution. Gross capacity $4 \times 10 = 40$ engineer-weeks; effective $40 \times 0.75 = 30$. Commitment ratio $66 \div 30 = 220\%$. Policy: commit to outcomes and ranges, not dated features; commit to about 70% of effective capacity (21 engineer-weeks) and hold the rest as buffer; require a published tradeoff for any addition; keep a dated promise only where there is a legal or contractual deadline. Retest: the share of committed items delivered, and the number of late date changes.

Alternative solution. Express the plan as "now, next, later" with confidence levels (high, medium, low) and update monthly.

Tradeoffs. Fewer promises give less certainty to sales and more reliability to the team; agree the cost with stakeholders.

EX-C29-05 · Scan your own artifacts

Problem. Run the anti-pattern scan (FW-C29-01) on three of your own artifacts (for example a spec, an update and your last two mock scorecards). Mark signals, count patterns, find the dominant cluster, choose one practice and set a retest date.

Context. Self-diagnosis.

Data. Your artifacts.

Constraints. Sixty minutes.

Expected approach. FW-C29-01 and FW-C29-02.

Solution. A good scan has a table of artifacts against signals, counts by pattern (for instance, "no outcome metric: 2 of 3 documents"; "no guardrail: 2 of 2 mocks"), a dominant cluster named with evidence, one practice with the artifact where it will appear, and a retest date with the signal to recheck. The practice is specific: "name the guardrail in the same sentence as the metric," not "be more rigorous." Share it with a colleague and ask them for one example of each pattern they have seen.

Alternative solution. Scan a teammate's roadmap with their permission as practice, using the catalog on evidence, and report signals, not labels.

Tradeoffs. A scan of three artifacts is quick and small; repeat it every quarter.

EX-C29-06 · A failure story in six parts

Problem. Using FW-C29-03, write a three-minute failure story for this scenario: you launched a notifications redesign that raised opens by 8% and opt-outs doubled; you reported the opens as success for six weeks before noticing the opt-outs.

Context. Interview use of the catalog.

Data. Invent two numbers and mark them (invented).

Constraints. Ten minutes.

Expected approach. FW-C29-03 and Worked example 5.

Solution. *What happened:* "I launched a notification redesign that raised opens by 8% and I reported it as a success; six weeks later I saw that daily opt-outs had risen from 0.6% to 1.2% (invented), which meant we were losing the ability to reach about 7,000 users a month (invented)." *Pattern:* optimizing the measure (cluster B): a single metric with no guardrail. *Root cause in my control:* I defined success as opens and never wrote a counter-metric. *Signal I missed:* opt-outs were in the dashboard from day one; I had not looked. *Practice changed:* every metric card now names a guardrail and I check both weekly; for notifications, opt-outs and unsubscribes are a launch gate. *Evidence:* in the next two notification changes we caught a rise in opt-outs in week one and rolled one back, avoiding about 4,000 lost reach (invented).

Alternative solution. Tell it as a post-mortem in writing, with the same six parts.

Tradeoffs. A precise failure story takes preparation and gives credibility.

EX-C29-07 · Pattern or deliberate choice?

Problem. For each decision say whether it is an anti-pattern or a defensible choice in context, and what evidence would decide: (a) a team launches to all users at once the week a law takes effect; (b) a PM builds a small internal tool before testing demand, because a test would take longer than the build; (c) a PM waits for a fix before telling leadership about a data error that affects a board report due in two days; (d) a team ships a feature with no outcome metric because it is a legal requirement.

Context. Applying the catalog with judgment.

Data. None.

Constraints. Six minutes.

Expected approach. C29.1 (when not to use the catalog).

Solution. (a) Defensible if the date is fixed by law and the scope is limited to what the law requires; the staged rollout is replaced by a very short, heavily monitored release with a rollback plan; evidence: the regulation's date and the cost of missing it. (b) Defensible when the cost of a build is below the cost of a test and the tool is reversible; still write the success measure. (c) An anti-pattern: bad news held back; tell leadership today with the impact and the plan, since the board report is the reason to tell, not to wait. (d) Defensible: a legal requirement needs a compliance measure (done, audited), not an outcome metric; record it as such. The test: what would the pattern's fix cost here, and what would it protect against?

Alternative solution. Write the condition under which each defensible choice would become an anti-pattern (for example, (b) if the tool is later used by customers).

Tradeoffs. Judging context takes more thought than applying a rule and avoids false alarms.

EX-C29-08 · Five whys to a cause you can change

Problem. The event: "the launch slipped three weeks." Write a chain of five whys that ends in a cause within your control, and the practice that would address it. Invent plausible links and mark them (invented).

Context. Root cause.

Data. None.

Constraints. Six minutes.

Expected approach. The five-whys model in section 9.

Solution. (1) Why did it slip? An integration failed late testing (invented). (2) Why late? Testing started two weeks before launch. (3) Why so late? The plan scheduled testing after the last feature was complete. (4) Why? We had no dependency list or early test of the riskiest integration. (5) Why?

I did not ask engineering for the riskiest technical assumption when I wrote the plan. The cause in my control is the last one. Practice: list the riskiest technical assumptions with engineering at kickoff and schedule a test of the top one in the first third of the project. Check: the share of projects where the top-risk test happened in the first third, and the number of late-found issues.

Alternative solution. Stop at the third why if the cause there is already something you can change, and say why.

Tradeoffs. Five questions can drift into blame; end at a choice you made, not at a person.

13. Practice protocol

Timed drills (three a week for two weeks).

- *Week 1: the catalog and the scan.* Use IQ-C29-01, IQ-C29-02, IQ-C29-06, EX-C29-01, EX-C29-02 and EX-C29-05. Scan three of your own artifacts and write your top pattern, its cluster and one practice.
- *Week 2: failure stories and diagnosis.* Use IQ-C29-03, IQ-C29-04, IQ-C29-05, IQ-C29-07, IQ-C29-08, EX-C29-03, EX-C29-04 and EX-C29-06. Write and rehearse a three-minute failure story with a named pattern and numbers, and practice the "why did this product fail" diagnosis aloud.

With a partner. Ask your partner to say "Isn't that just bad luck?" and "How do you know the fix works?" at random.

Feedback checklist (yes, partly, no).

1. Did I name the pattern, not just the event?
2. Is my root cause a choice I made?
3. Did I name a measurable signal that was available?
4. Is my fix a practice with an artifact and a check?
5. Did I give a before-and-after number?
6. Did I scan across artifacts, not one data point?
7. Did I avoid labeling other people's decisions?

8. Did I ask whether context made the pattern deliberate?

Answer journal. Record the date, the prompt, the pattern, the signal, the practice and the number, your score and the one fix.

14. Implementation playbook

Apply the chapter to your own work over three weeks.

Days 1 to 3. Collect three artifacts (a spec or proposal, a status update, a roadmap or backlog with outcome measures) and the last two mock scorecards if you have them.

Week 1. Run the scan (FW-C29-01): mark signals, count patterns, find the dominant cluster. Ask one teammate for one example of each pattern they have seen in your work.

Week 2. Choose one practice for the dominant cluster and apply it to the next artifacts. Write your failure story (FW-C29-03) for the pattern you found, with a number.

Week 3. Retest the signal on the new artifacts. If it has changed, write the before-and-after; if not, change the practice and retest again.

Deliverables that become stories: the scan table (evidence that you diagnose yourself), the practice and its retest result (a changed habit with a number), and the failure story (a ready answer to the commonest behavioral question). Keep every number real and credit people who helped you see the pattern.

15. Key takeaways and further reading

Key takeaways

1. Anti-patterns are common, sensible-looking responses that produce worse results; most grow from five roots: activity mistaken for progress, optimizing the measure, avoiding discomfort, pattern-matching instead of reasoning, and not closing the loop.
2. Every anti-pattern has a detection signal you can measure: outcome coverage and shipped-and-measured share for a feature factory, a distribution behind a headline number, committed work against effective capacity, guardrail errors across mocks.
3. Use the catalog on yourself: scan your own artifacts, count patterns, find the dominant cluster, choose one practice and retest; do not label other people's decisions.
4. Tell a failure as a pattern with a root cause in your control, a missed signal, a changed practice and evidence it works; a fix is credible only as a habit with an artifact and a check.
5. Check whether context made a pattern deliberate (a legal deadline can justify a big-bang launch; a costly test can justify building first) before calling it a mistake.

Further reading

- Google's posts of 8 October 2018 ("Project Strobe") and 10 December 2018 ("Expediting changes to Google+"), and Google Workspace's post of 30 January 2019. Sources for CS-C29-02 (confirmed in Phase 5, October 2026).
- Marty Cagan, *Inspired* (2017), and Melissa Perri, *Escaping the Build Trap* (2018), on output-focused teams and outcome-focused alternatives [V: Cagan: 2nd edition, Wiley, December 2017; Perri: O'Reilly, November 2018].
- Charles Goodhart's observation on measures becoming targets, as discussed in management writing, and the references in C05 [V: Goodhart 1975; Strathern 1997].
- Chapters C05, C06, C15, C16, C17, C26, C31, C32 and C34 of this book, which teach the fixes this chapter points to.